

Python od podstaw: Implementuj własne funkcje statystyczne

Zestaw 10 zadań programistycznych dla studentów data science, którzy chcą nauczyć się programować i lepiej zrozumieć, jak działają biblioteki NumPy i Pandas. Każde zadanie to implementacja popularnej funkcji statystycznej napisana czystym Pythonem, bez importowania gotowych rozwiązań.

```
1
2 import pandas as pd
3 import numpy as np
4
5 {
6     fFebnm stnadttin Prefents" {
7     {
8         Wpart sesan{"esno:
9         fdtalc exane = neomne"vafob",
10        ipanc istdr";
11        apape sctant_i06"vatnictionor ("S77B4");
12    }
13 }
14
15 from sklearn.model_selection import train_test_split
16
17 def test_data: {
18     celes: "70ns1" c" );
19     ceters: ("anpt_sonipnctes (186" AP");
20 }
21
22 Predictions vs Actual {
23 }
24
25 }
26
27 }
28
29 }
```

Przegląd zadań

Poniżej znajdziesz wszystkie 10 funkcji do zaimplementowania. Zadania są ułożone od najprostszych do bardziej złożonych — każde kolejne może (i powinno!) korzystać z poprzednich.

01

our_sum

Suma elementów listy — jak wbudowane `sum()`

02

our_mean

Średnia arytmetyczna — jak `np.mean()`

03

our_std

Odchylenie standardowe — jak `np.std()`

04

our_median

Mediana — jak `np.median()`

05

our_quartile

Kwartyle Q1 i Q3 — jak `np.percentile()`

01

our_max

Maksimum — jak `max()` lub `np.max()`

02

our_min

Minimum — jak `min()` lub `np.min()`

03

our_describe

Statystyki opisowe — jak `df.describe()`

04

our_value_counts

Liczenie wystąpień — jak `Series.value_counts()`

05

our_groupby

Grupowanie i agregacja — jak `df.groupby()`

Zadanie 1 i 2: our_sum i our_mean

📌 Zadanie 1 — our_sum

Napisz funkcję `our_sum(data)`, która przyjmuje listę liczb i zwraca ich sumę. **Nie używaj** wbudowanej funkcji `sum()`. Użyj pętli `for` i akumulatora.

```
# Dane wejściowe:  
data = [2, 4, 6, 8]  
  
our_sum(data)  
# Output: 20
```

📌 Wskazówka: zacznij od zmiennej `result = 0` i dodawaj kolejne elementy w pętli.

📌 Zadanie 2 — our_mean

Napisz funkcję `our_mean(data)`, która oblicza średnią arytmetyczną listy. **Możesz** — a nawet powinieneś — skorzystać z `our_sum()` napisanej w zadaniu 1. Sprawdź wynik używając `np.mean(data)`.

```
# Dane wejściowe:  
data = [2, 4, 6, 8]  
  
our_mean(data)  
# Output: 5.0  
  
import numpy as np  
np.mean(data)  
# Output: 5.0 ✓
```

📌 Wzór: $\text{mean} = \text{suma} / \text{liczba_elementów}$. Użyj `len(data)` do pobrania długości listy.

Zadanie 3 i 4: our_std i our_median

Zadanie 3 — our_std

Odchylenie standardowe to miara rozproszenia danych wokół średniej. Implementacja wymaga kilku kroków: obliczenia średniej, wyznaczenia odchyłeń od niej, podniesienia ich do kwadratu, uśrednienia i wyciągnięcia pierwiastka. Skorzystaj z `our_mean()`.

```
# Dane wejściowe:
data = [2, 4, 4, 4, 5, 5, 7, 9]

our_std(data)
# Output: 2.0

np.std(data)
# Output: 2.0 ✓
```

 Wzór: `std = sqrt(mean((x - mean(x))^2))`. Do pierwiastka użyj `x ** 0.5`.

Zadanie 4 — our_median

Mediana to wartość środkowa posortowanego zestawu danych. **Krok 1:** posortuj listę (`sorted()` jest dozwolone). **Krok 2:** sprawdź czy liczba elementów jest parzysta czy nieparzysta. Dla nieparzystej — zwróć środkowy element. Dla parzystej — zwróć średnią dwóch środkowych.

```
# Nieparzysta liczba elementów:
data_odd = [3, 1, 4, 1, 5]
our_median(data_odd)
# Output: 3.0 (posortowane: [1,1,3,4,5])

# Parzysta liczba elementów:
data_even = [1, 3, 5, 7]
our_median(data_even)
# Output: 4.0 (średnia z 3 i 5)

np.median(data_even)
# Output: 4.0 ✓
```

Zadanie 5: `our_quartile`

To najtrudniejsze z zadań dotyczących statystyk opisowych. Kwartyle dzielą dane na cztery równe części — Q1 to mediana dolnej połowy, Q3 to mediana górnej połowy. Możesz wewnątrz funkcji `our_quartile` obliczyć medianę funkcją `our_median` i podzielić dane na dolną połowę - mniejsze od mediany, oraz górną połowę - większe od mediany. Wtedy, w zależności od tego, czy użytkownik podał jako parametr pierwszy, czy trzeci kwartył, możesz zwrócić `our_median( dolna połowa )` lub `our_median( górna połowa )`.

Zadanie 6 i 7: our_max i our_min

Te dwa zadania są prostsze, ale uczą ważnej techniki programistycznej — iterowania po liście z zapamiętywaniem bieżącego ekstremum. **Nie używaj** wbudowanych funkcji `max()` i `min()`.

Zadanie 6 — our_max

Napisz funkcję `our_max(data)`, która zwraca największą wartość w liście. Zainicjuj zmienną pierwszym elementem listy, a następnie porównuj każdy kolejny element z aktualnym maksimum.

```
# Dane wejściowe:  
data = [3, 7, 2, 9, 1, 5]  
  
our_max(data)  
# Output: 9  
  
# Weryfikacja:  
max(data) # Output: 9 ✓  
np.max(data) # Output: 9 ✓
```

Zadanie 7 — our_min

Napisz funkcję `our_min(data)`, analogiczną do `our_max`, ale zwracającą najmniejszą wartość. Pamiętaj: zainicjuj zmienną pierwszym elementem, nie zerem — bo co jeśli wszystkie wartości są dodatnie i większe od zera?

```
# Dane wejściowe:  
data = [3, 7, 2, 9, 1, 5]  
  
our_min(data)  
# Output: 1  
  
# Weryfikacja:  
min(data) # Output: 1 ✓  
np.min(data) # Output: 1 ✓
```

  Bonus: spróbuj napisać obie funkcje jako jedną, z parametrem `mode='max'` lub `mode='min'`. Jak zmieni się kod?

Zadanie 8: our_describe

To zadanie spinające wszystkie poprzednie. Funkcja `our_describe(data)` ma zwracać słownik z kluczowymi statystykami opisowymi — dokładnie jak `df.describe()` w Pandas. **Musisz wykorzystać wszystkie wcześniej napisane funkcje.** To świetny przykład kompozycji funkcji w praktyce.

Wymagane klucze słownika

- `count` — liczba elementów (`len()`)
- `mean` — wynik `our_mean()`
- `std` — wynik `our_std()`
- `min` — wynik `our_min()`
- `25%` — wynik `our_quartile(..., 1)`
- `50%` — wynik `our_median()`
- `75%` — wynik `our_quartile(..., 3)`
- `max` — wynik `our_max()`

Przykład użycia

```
data = [1, 2, 3, 4, 5, 6, 7, 8]
```

```
our_describe(data)
```

```
# Output:
```

```
# {
```

```
# 'count': 8,
```

```
# 'mean': 4.5,
```

```
# 'std': 2.29,
```

```
# 'min': 1,
```

```
# '25%': 2.5,
```

```
# '50%': 4.5,
```

```
# '75%': 6.5,
```

```
# 'max': 8
```

```
# }
```

```
# Weryfikacja:
```

```
import pandas as pd
```

```
pd.Series(data).describe()
```

```
# count    8.000000
```

```
# mean     4.500000
```

```
# std      2.449490
```

```
# min      1.000000
```

```
# 25%      2.750000
```

```
# 50%      4.500000
```

```
# 75%      6.250000
```

```
# max      8.000000
```

📌 Różnica w `std` między naszą implementacją a Pandas wynika z podziału przez `n` (populacja) vs `n-1` (próba). To tzw. korekta Bessela — wróćmy do tego na zajęciach ze statystyki!

Zadanie 9: our_value_counts

Funkcja `our_value_counts(data)` powinna zwracać słownik, w którym klucze to unikalne wartości z listy, a wartości to liczba ich wystąpień — posortowany malejąco według liczby wystąpień. Wzorujemy się na `Series.value_counts()` z Pandas.

Algorytm

1 Utwórz pusty słownik

Będzie przechowywał liczniki dla każdej wartości.

2 Iteruj po liście

Dla każdego elementu: jeśli istnieje w słowniku — zwiększ licznik o 1. Jeśli nie — dodaj z wartością 1.

3 Posortuj malejąco

Zwróć słownik posortowany po wartościach (licznikach) malejąco. Użyj `sorted()` z parametrem `reverse=True`.

Alternatywnie możesz każdej unikalnej wartości listy przypisać w słowniku wynik ***`nasza_lista.count(unikalna_wartosc)`***

Zadanie 10: our_groupby

```
df = pd.DataFrame({  
    "num": [12, 7, 15, 3, 9, 21, 14, 6, 18, 11, 5, 17],  
    "cat": ["A", "B", "C", "A", "B", "C", "A", "B", "C", "A", "B", "C"]  
})
```

df

	num	cat
0	12	A
1	7	B
2	15	C
3	3	A
4	9	B
5	21	C
6	14	A
7	6	B
8	18	C
9	11	A
10	5	B
11	17	C

```
df.groupby('cat')['num'].sum()
```

```
cat  
A    40  
B    27  
C    71  
Name: num, dtype: int64
```

```
our_groupby( df, cat_name = 'cat', num_col_name = 'num', compute = 'sum' )
```

```
A    40  
B    27  
C    71  
dtype: int64
```

Napisz funkcję, która daje taki sam output jak metoda *groupby* w *pandas*.

Podsumowanie

Każdą zaimplementowaną funkcję powinieneś przetestować na podanych przykładach oraz własnych danych .

✓ Testuj na znanych danych

Używaj małych, ręcznie obliczalnych zestawów danych (np. [2, 4, 6, 8]) — łatwo sprawdzisz wynik w głowie zanim porównasz z NumPy/Pandas.

10

Funkcji do zaimplementowania

↻ Kompozycja funkcji

Zadania od 5 do 10 wymagają użycia wcześniejszych funkcji. Jeśli `our_mean` jest błędna, `our_std` i `our_describe` też będą błędne — naprawiaj od podstaw.

4

Zadania z kompozycją funkcji

`our_quartile`, `our_describe`, `our_groupby`,
`our_std`

3

Biblioteki do weryfikacji

Python built-ins, NumPy, Pandas